REQDECK

Your first API requests

A friendly guide to ReqDeck for people who have never sent a request.

GET
/posts/1
200 OK
{ "id": 1,
"title": "…" }

What is inside

1 Your first request in five minutes

2 What a request is made of

3 Variables and environments

4 Logging in: authentication

5 Reading the answer

6 History and cURL

7 Your data

8 When something does not work

★ Keys worth knowing

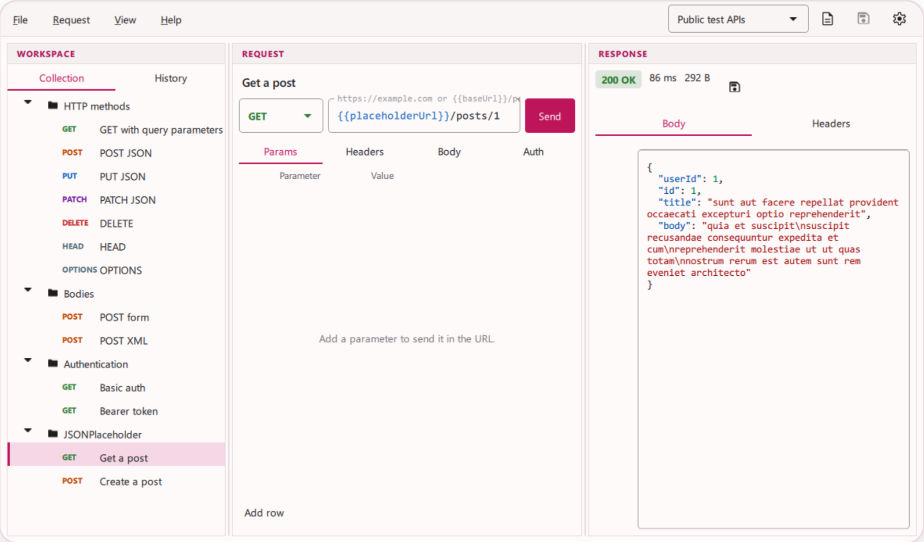
↗ Places to practice

CHAPTER 1

Your first request in five minutes

A request is a message your computer sends to a server. The server answers with a response. That is all an API is: a server that answers requests from programs instead of from people with a browser.

1. Start ReqDeck and choose Open Samples.
2. ReqDeck opens the request "Get a post" from the folder JSONPlaceholder in the Collection on the left.
3. Look at the address in the middle: this is where the request goes.
4. Press Send, or `Ctrl+Enter`.
5. On the right you see the answer: a green 200 OK and the post as JSON.

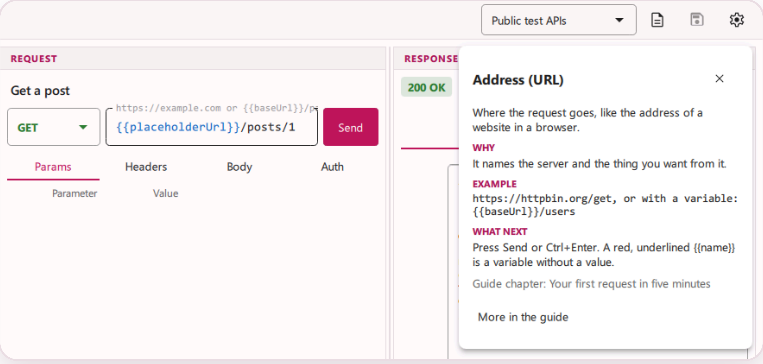

File
Request
View
Help
Public test APIs
WORKSPACE
Collection
History
HTTP methods
GET GET with query parameters
POST POST JSON
PUT PUT JSON
PATCH PATCH JSON
DELETE DELETE
HEAD HEAD
OPTIONS OPTIONS
Bodies
POST POST form
POST POST XML
Authentication
GET Basic auth
GET Bearer token
JSONPlaceholder
GET Get a post
POST Create a post
REQUEST
Get a post
GET
https://example.com or {{baseUrl}}/p
{{placeholderUrl}}/posts/1
Send
Params
Headers
Body
Auth
Parameter
Value
Add a parameter to send it in the URL.
Add row
RESPONSE
200 OK
86 ms
292 B
Body
Headers

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident
occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit
recusandae consequuntur expedita et
cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem
eveniet architecto"
}
```

A request in the middle, its answer on the right.

Tip

Not sure what something is? Point at it and press `F1`.


Public test APIs
REQUEST
Get a post
GET
https://example.com or {{baseUrl}}/p
{{placeholderUrl}}/posts/1
Send
Params
Headers
Body
Auth
Parameter
Value
RESPONSE
200 OK
×

Address (URL)

Where the request goes, like the address of a website in a browser.

WHY

It names the server and the thing you want from it.

EXAMPLE

`https://httpbin.org/get, or with a variable: {{baseUrl}}/users`

WHAT NEXT

Press Send or Ctrl+Enter. A red, underlined {{name}} is a variable without a value.

Guide chapter: Your first request in five minutes

More in the guide

Press `F1` and ReqDeck explains what is under the pointer.

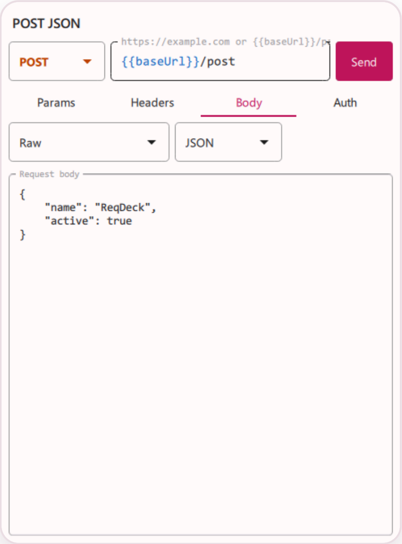
CHAPTER 2

What a request is made of

Think of a request as a letter. The method is what you ask for, the address is where it goes, parameters and headers are notes on the envelope, and the body is the letter inside.

GET	Read something. No body.
POST	Create something new. Usually with a body.
PUT / PATCH	Change something that exists.
DELETE	Remove something.

Parameters are added to the address after a question mark, for example ?page=2. Headers carry information about the request, such as the format you accept. The body carries data, most often as JSON.



POST JSON

POST

`{{baseUrl}}/post`

Raw

JSON

Request body

```
{
    "name": "ReqDeck",
    "active": true
}
```

A JSON body for a POST request.

Tip

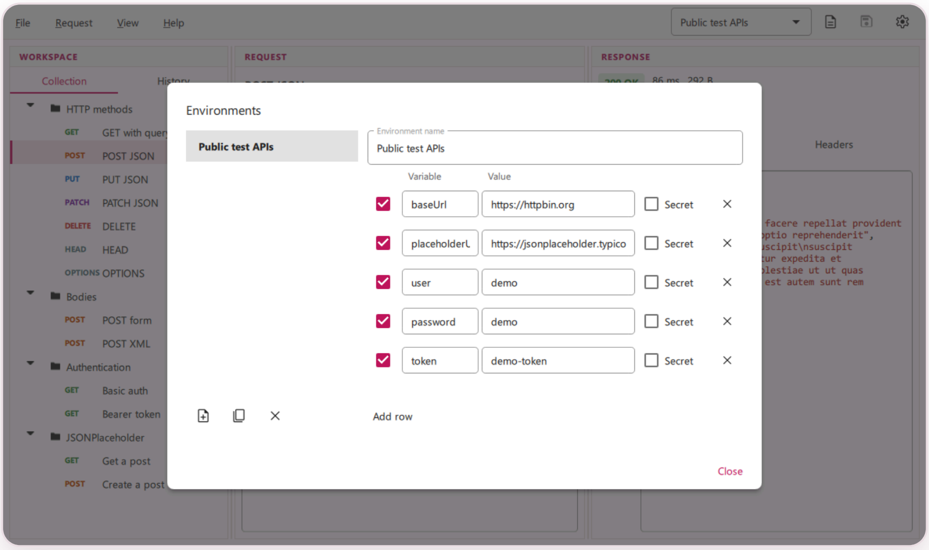
`Ctrl+K` finds any request by a few words of its name or address, and any menu command. In the tree, Ctrl+click or Shift+click selects several items, F2 renames, Delete deletes, and right-click > Sort A–Z orders a folder.

CHAPTER 3

Variables and environments

A variable is a name for a value you use in many requests. Write `{{baseUrl}}` instead of the full server address, and change the address in one place.

An environment is a set of variables. Make one for each server, for example Staging and Production, and switch between them at the top of the window.


File
Request
View
Help
Public test APIs
WORKSPACE
Collection
History
REQUEST
RESPONSE
HTTP methods
GET with query
POST JSON
PUT JSON
PATCH JSON
DELETE
HEAD
OPTIONS
Bodies
POST form
POST XML
Authentication
Basic auth
Bearer token
JSONPlaceholder
Get a post
Create a post
Headers
Add row
Close

Environments

Public test APIs

Environment name: Public test APIs

	Variable	Value		
☑	baseUrl	https://httpbin.org	☐ Secret	×
☑	placeholderL	https://jsonplaceholder.typico	☐ Secret	×
☑	user	demo	☐ Secret	×
☑	password	demo	☐ Secret	×
☑	token	demo-token	☐ Secret	×

Environments and their variables.

Tip

A `{{name}}` in red with an underline has no value in the active environment.

Careful

Tick Secret for passwords and keys. Their values are then kept out of the workspace file.

Tip

A field that names a variable the active environment does not have gets a red frame. Point at the field to see which variable is missing.

Tip

Type {{ and ReqDeck lists the variables of the active environment; Enter inserts one. If a request uses a variable the environment lacks, Send asks for its value first.

CHAPTER 4

Logging in: authentication

Many APIs answer only to users they know. The Auth tab adds the proof to each request.

Basic	User name and password.
Bearer	A token the service gave you.
API key	A key, sent as a header or a parameter.

> **Careful**
>
> Use variables such as `{{token}}` with Secret ticked, so the value is not saved in the workspace file.

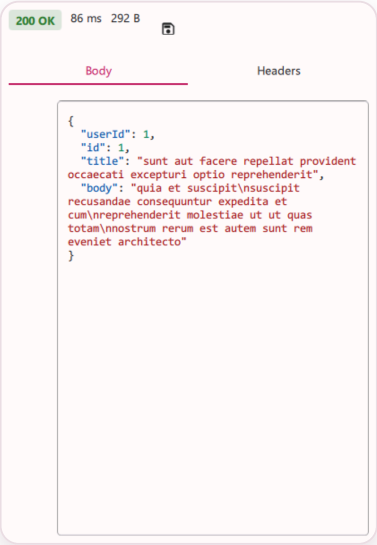
> **Tip**
>
> A password, token, or key you type into a request is saved in the workspace file. ReqDeck then offers Move to a secret: the value goes into a secret variable, encrypted on this computer, and the field shows `{{token}}` instead.

CHAPTER 5

Reading the answer

The first number tells whether the request worked. Codes starting with 2 mean success, with 4 a problem with the request, with 5 a problem on the server.

200 OK	It worked.
201 Created	Something new was created.
204 No Content	It worked; there is nothing to show.
400 Bad Request	The server did not understand the request. Check the body and parameters.
401 Unauthorized	You are not logged in, or the token expired.
403 Forbidden	You are logged in but not allowed.
404 Not Found	Nothing at this address. Check the URL.
429 Too Many Requests	Slow down and try later.
500 Internal Server Error	The server failed. Not your fault, usually.



```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident
occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit
recusandae consequuntur expedita et
cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem
eveniet architecto"
}
```

Status, time, size, and the body of the answer.

Tip

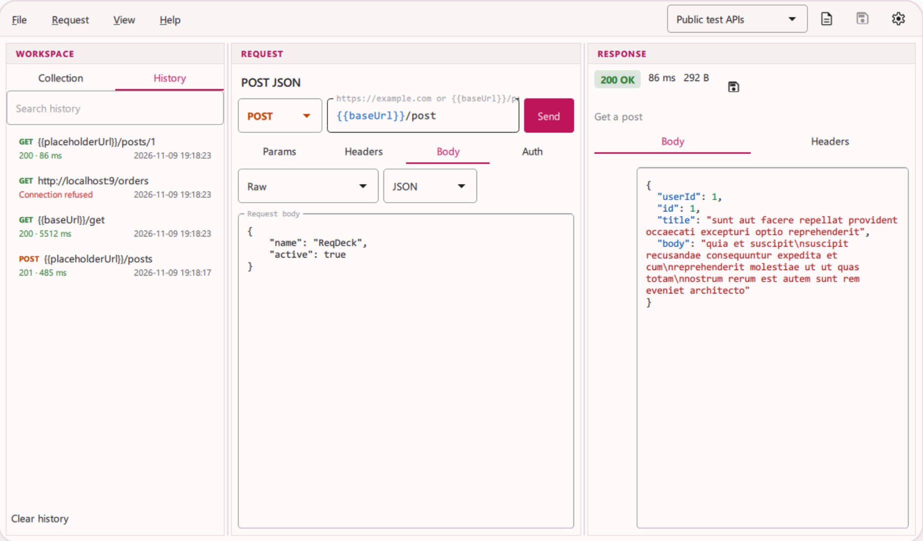
`Ctrl+F` finds text in the answer. The save button beside the size writes the answer to a file, also a very large one, of which only the start is shown.

CHAPTER 6

History and cURL

Every request you send lands in History, on the left. Double-click an entry to make a new request from it. Passwords and tokens written in a request are not kept there.

Documentation often shows requests as curl commands. Use File > Import cURL to turn one into a request, and Request > Copy as cURL to share yours.


File
Request
View
Help
Public test APIs
WORKSPACE
Collection
History
Search history
Clear history
REQUEST
POST JSON
POST
https://example.com or {{baseUrl}}/p
Send
Params
Headers
Body
Auth
Raw
JSON
Request body
RESPONSE
Body
Headers

```
GET {{placeholderUrl}}/posts/1
200 · 86 ms                2026-11-09 19:18:23
GET http://localhost:9/orders
Connection refused         2026-11-09 19:18:23
GET {{baseUrl}}/get
200 · 5512 ms              2026-11-09 19:18:23
POST {{placeholderUrl}}/posts
201 · 485 ms               2026-11-09 19:18:17
```

`{{baseUrl}}/post`

```
{
    "name": "ReqDeck",
    "active": true
}
```

200 OK 86 ms 292 B

Get a post

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

History with the status of each request.

Tip

The search field above the history keeps the entries that contain every word you type, for example post orders.

CHAPTER 7

Your data

Your requests live in the workspace file you chose, a .reqdeck file you may copy or share. Secret values of environments are not in it: ReqDeck keeps them encrypted for your Windows user on this computer.

On another computer, or under another Windows user, the secret values are empty. ReqDeck says so when the workspace opens. Enter them again and save.

Before a new version changes the format of these files, ReqDeck keeps a copy of the workspace file and its secret values. File > Restore Previous Copy puts them back; changes made since then are lost.

> **Tip**
>
> Two ReqDeck windows can run at once: both add to the same history.

CHAPTER 8

When something does not work

Connection refused	Nothing answers at this address. Is the server running? Is the port right?
Host not found	The server name is wrong or the network is down.
Timeout	The server took too long. Raise the timeout or try later.
Unresolved variable	A `{{name}}` has no value. Pick the right environment.
Certificate error	The server's HTTPS certificate is not trusted, common on test servers. If you know the server, press Continue.
Proxy or company network	ReqDeck uses the proxy set in Windows, like your browser. Requests to your own computer always go directly.
Secret values are empty on another computer	Secret values are encrypted for your Windows user on the computer where you saved them. On another computer or user, enter them again and save. The workspace file itself is not affected.

Tip

View > Show Log Panel shows what ReqDeck did and what went wrong.

Keys worth knowing

`F1`	Explain what is under the pointer or in the field you type in.
`Ctrl` + `Enter`	Send the request.
`Ctrl` + `K`	Find a request or a command.
`Ctrl` + `S`	Save the workspace.
`Ctrl` + `N`	Add a new request.
`Ctrl` + `O`	Open a workspace.
`F2`	Rename the selected item in the tree.
`Esc`	Close the help or a dialog.

Places to practice

https://httpbin.org
A free public API that answers whatever you send. Safe to experiment on.

https://developer.mozilla.org/docs/Web/HTTP/Status
Every HTTP status code explained.